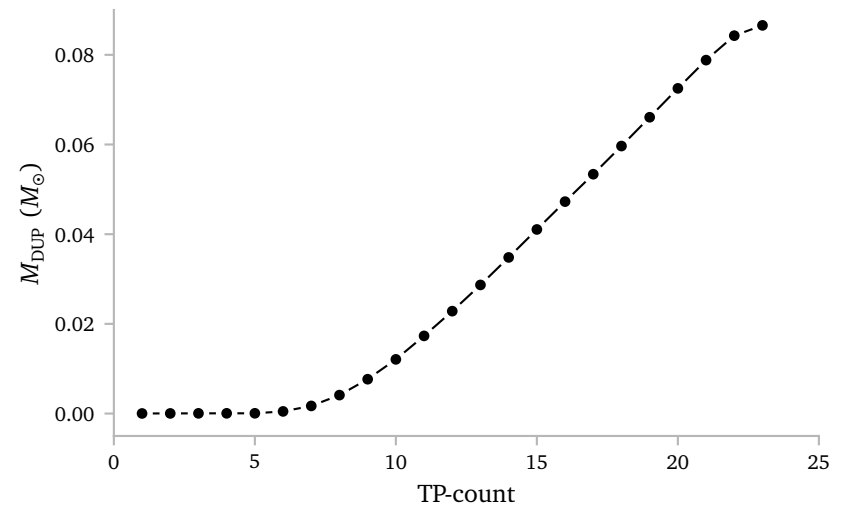


Notes Week 22

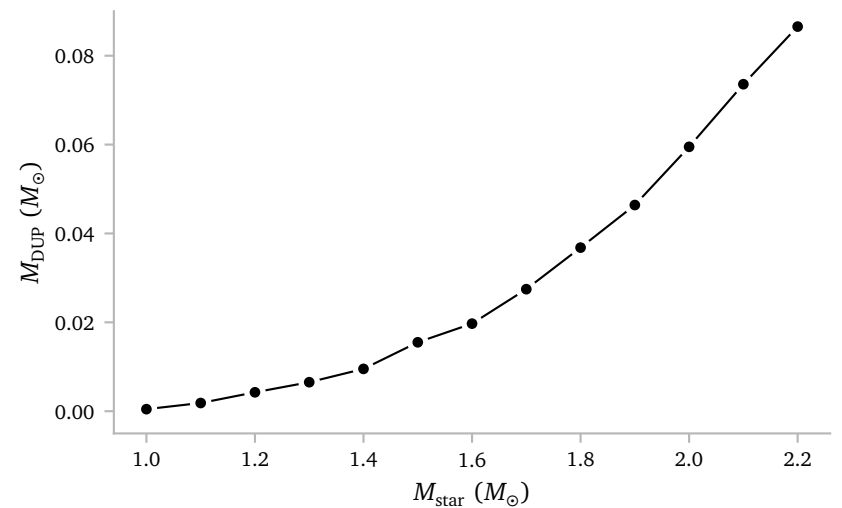
a. Dredge-Up mass

Since the current plan is to use the amount of mass that has been dredged up and the intershell abundances to infer the envelope abundances, I must be sure that I can actually accurately get this dredged up mass.

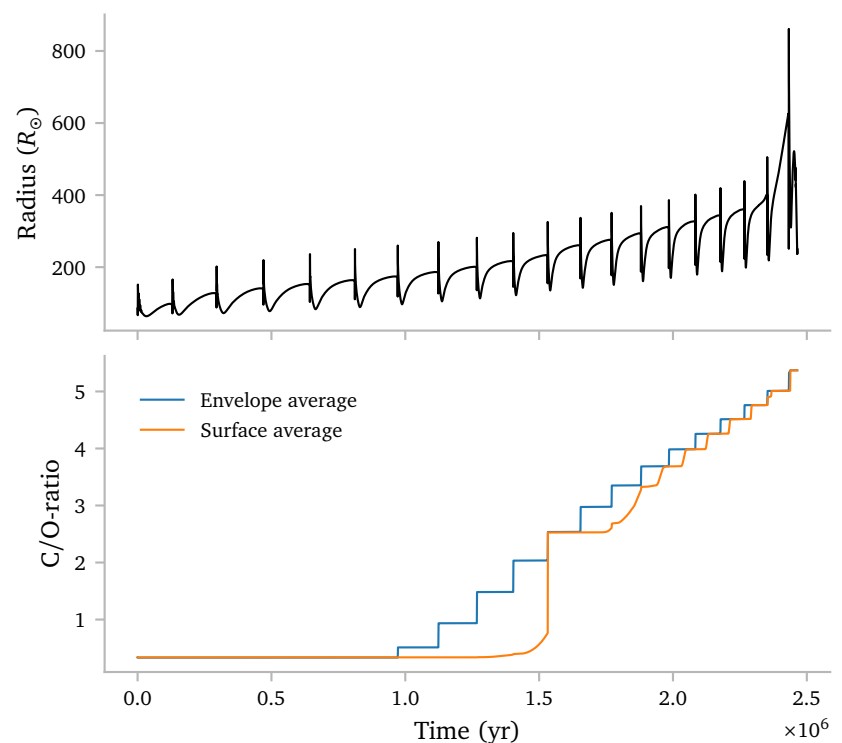
Below I show that this is indeed possible, and the values align roughly with what I would expect from Figure 1 of [Karakas & Lugaro \(2016\)](#).



Below, I plot the total amount of dredged-up mass as a function of initial star age.



b. Envelope Abundances



c. Dwarf Metallicity

I am using $[\text{Fe}/\text{H}] = -0.4$ which equates to $z \approx 0.00557$.

d. Extended Grid

d.1 Idea

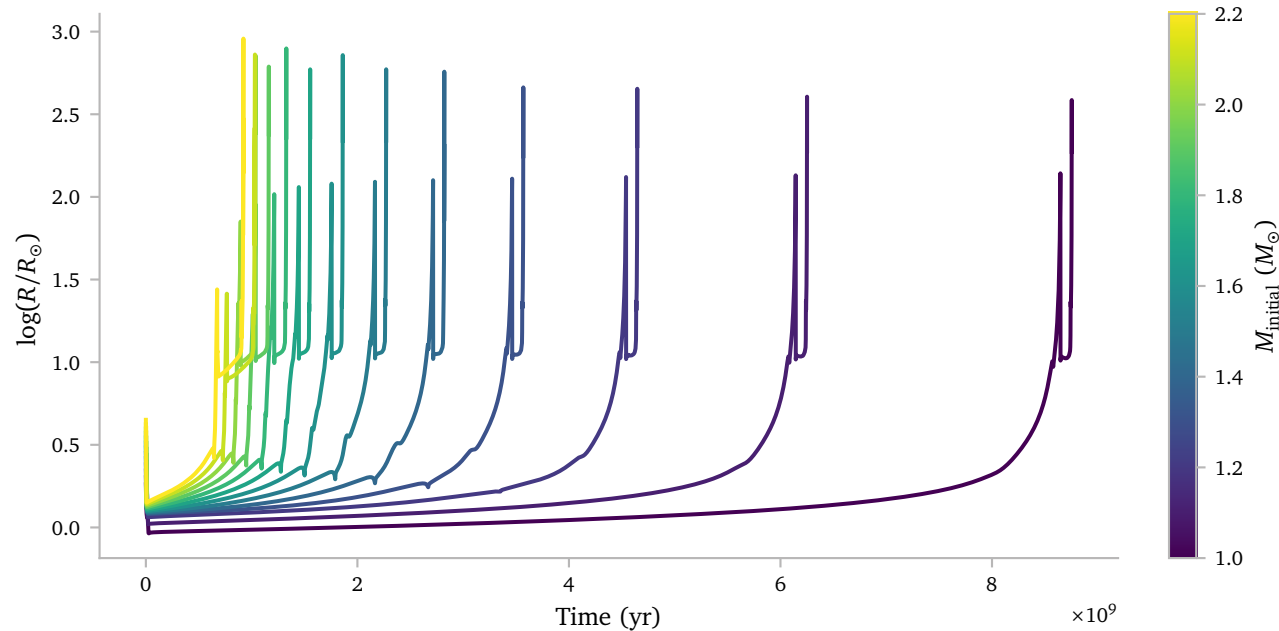
I want to simulate TPAGB stars from 1 to 3.5 $M_{\odot}$ in order to make sure I get the minimum mass for TPAGB stars.

Initially, I use a dense spacing of $\Delta M = 0.1 M_{\odot}$. This is probably a bit much to construct grids with, but at least I can clearly find the starting mass of the TPAGB, and I can see the stability of a wide range of masses.

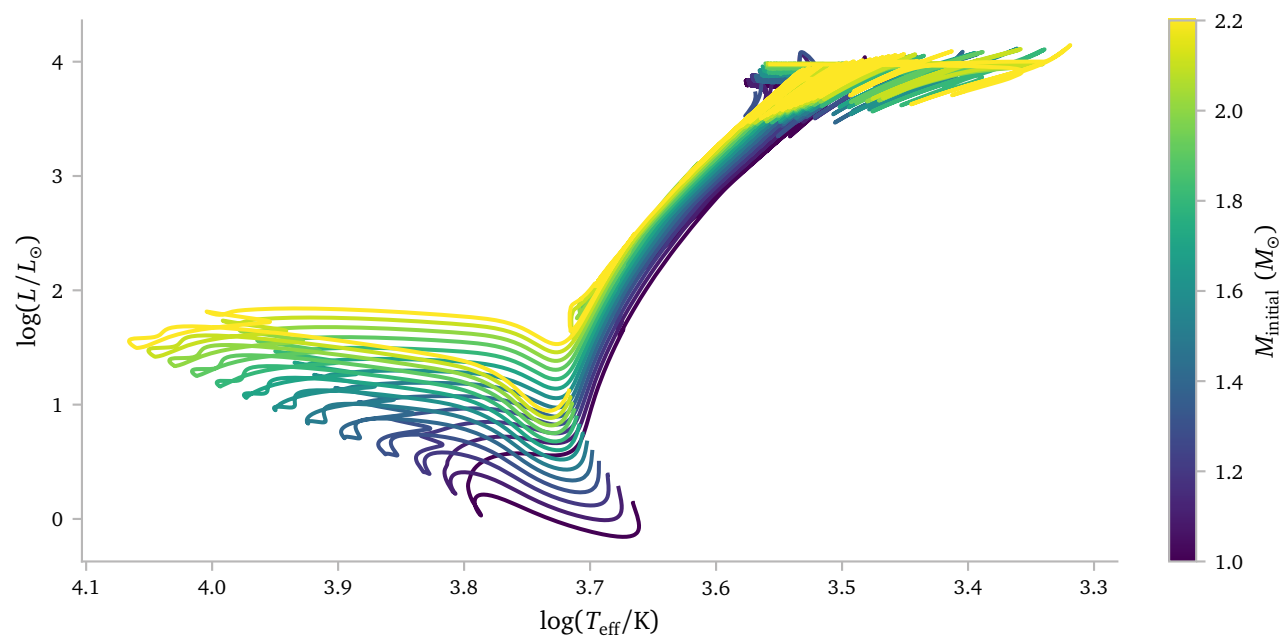
e. Results

It turns out that MESA does have quite a bit of trouble simulating single star TPAGB stars beyond something like 2 $M_{\odot}$. Especially the last pulses seem to cause a lot of trouble with convergence.

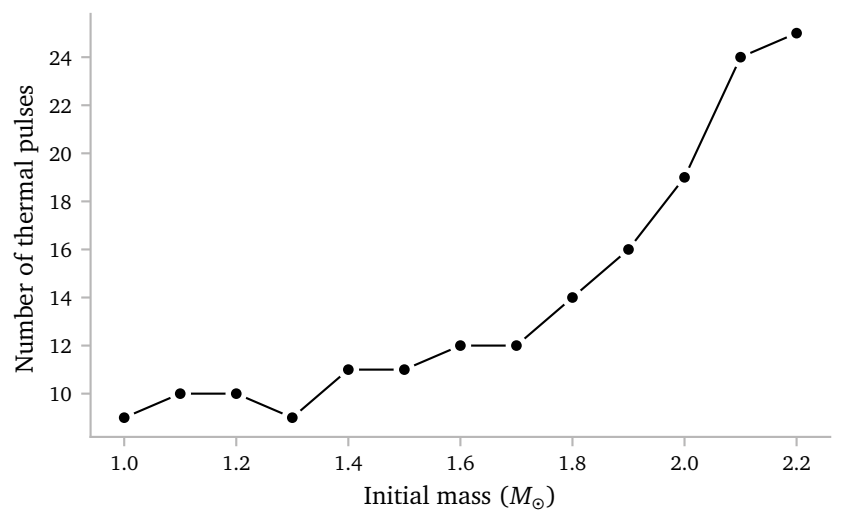
Below I plot the models of the stars from $M = 1 M_{\odot}$ to $M = 2.2 M_{\odot}$.



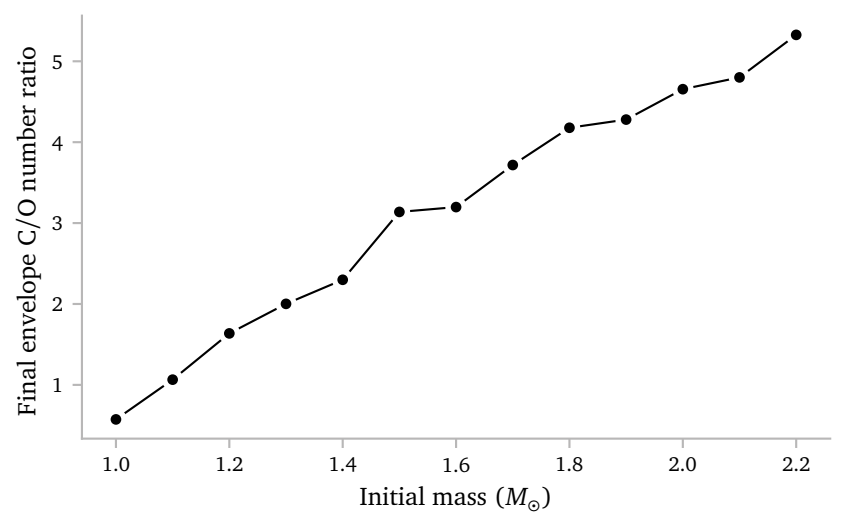
This looks good. I expect the lifetime of heavier stars to be shorter and that is exactly what I observe. We also see that the heavier stars reach larger radii, which is also expected.



The HR also looks like what I would expect. The heavier stars are more towards the blue and luminous side on the MS. On the Giant branches, the stars are on the Hayashi tracks, where the approximate temperature is determined by the mass of the stars. Again, the results are as I would expect.



We see that the number of thermal pulses increases with mass. This is in line with the results from various studies of TPAGB-stars (eg. [Rees et al., 2024](#); [Karakas et al., 2002](#); [Karakas & Lugaro, 2016](#)).



The CO-ratio also increases with mass, which makes sense, since we saw that the amount of dredged up mass increases with mass too. We can almost directly compare these results with the results from [Karakas & Lugaro \(2016\)](#). They use a metallicity of 0.007, so slightly higher, and they find very similar C/O-ratios.

f. Optimizing Grid Generation Code

f.1 Idea

Since we will be dealing with much larger grids and more masses, I think it is very much time to optimize the grid generation code. Up to now, it would read in the compute star model to use as input. This is very slow.

My idea is to create some optimized binary file that contains only the necessary data for the grid generation.

I also want to be able to generate more general grids, where we can vary any parameter, not just the initial Roche lobe radius and mass ratio but also for example the initial mass or the amount of accretion.

I also want to generate some settings.json files so I can easily inspect which which settings the grid was generated, as well as what the settings of individual models are.

The code I used previously also could drastically use a refactor

f.2 Implementation

The code now neatly starts with the user variables:

```
# WARNING: CHECK / MODIFY THESE PATHS
grid_name = "epsilon-grid-test"
proj_dir = "/home/koen/master-internship"
reference_binary_dir =
↳ f"{proj_dir}/mesa-models/reference-binary/2026-07-01/"
binary_exe_dir =
↳ f"{proj_dir}/mesa-models/reference-binary/"

# WARNING: SETTINGS FOR THE GRID
single_star_dirs =
↳ [f"{proj_dir}/mesa-models/single-stars/z0.00557/completed/M1.5"]
Rs = [350]
qs = np.linspace(0.4, 1, 7)
epss = np.linspace(0, 0.85, 3)

mass_transfer_beta = np.linspace(0, 1, 7)
mass_transfer_delta = 1 - mass_transfer_beta

# WARNING: SETTINGS FOR BINARY SIMULATION
mass_transfer_alpha = 0.0
mass_transfer_gamma = 1.44
```

I then do some general setup things like creating the folder, setting some paths and getting the metallicity.

```
grid_dir = f"{proj_dir}/mesa-models/{grid_name}/"
models_dir = f"{single_star_dirs[0]}/models/"
os.makedirs(os.path.dirname(grid_dir), exist_ok=True)
shutil.copyfile(f"{binary_exe_dir}/binary",
↳ f"{grid_dir}/binary")
```

```
def get_initial_Z():
    model = [os.path.basename(f) for f in
↳ os.scandir(models_dir)][0]
    Z = mr.MesaData(f"{models_dir}/{model}").initial_z
    return Z
```

star_Z = get_initial_Z()

For the rest of process, I use two helper classes:

```
class Evolution:
    def __init__(self, bin, Star, inds):
        self.bin = bin
        self.star = Star
        self.inds = inds

class GridPoint:
    def __init__(self, evolution, model):
        self.evolution = evolution
        self.model = model
```

I then have the following functions to create the setting files:

```
def generate_settings_file():
def generate_run_file(
    R, q, model_name, age, m1, m2, a, omega, varcontrol,
↳ beta, delta, eps, run_dir
):
```

I call generate_settings_file() immediately and use the generate_run_file() only during the grid generation process.

The grid generation loop looks like this:

```
for single_star_dir in single_star_dirs:
    for R in Rs:
        for q in qs:
            print(R, q)
            base = {
                "R": R,
                "q": q,
                "single_star_dir": single_star_dir,
            }

            grid_point = prepare_grid_point(params=base)

            if grid_point is None:
                print("no suitable model")
                continue
            if grid_point is False:
                print("skipping this R")
                break

            for eps in epss:
                if eps == 1:
                    params = {
                        **base,
                        "beta": 0,
                        "delta": 0,
                        "eps": eps,
                    }

                    generate_run(params=params,
↳ grid_point=grid_point)
                    continue

                for beta, delta in
↳ zip(mass_transfer_beta,
↳ mass_transfer_delta):
                    params = {
                        **base,
                        "beta": beta * (1 - eps),
                        "delta": delta * (1 - eps),
                        "eps": eps,
                    }

                    generate_run(params=params,
↳ grid_point=grid_point)
```

These are the three variables that change the simple integration outcome from the evolve.py script that I am using. I split these from the other variables to prevent unnecessary double computation.

Here, I do the bulk of the computation, including the simple integration.

Next there is some logic to stop the generation of the current R if a model failed already

Finally, I add the other variables that do not change the computation, only the MESA settings and I change all of the necessary files.

This all happens here.

For completeness, This is what prepare_grid_point() looks like:

```
def prepare_grid_point(params):

    evolution = evolve_binary(params)
    if evolution is None:
        return None

    contact_age = find_contact(evolution)
    if contact_age is None:
        return None

    if contact_age is False:
        return False

    models_dict = get_model_dict(params)
    model = select_model(contact_age, models_dict)
    if model is None:
        return None

    return GridPoint(evolution, model)
```

and this is what generate_run() looks like:

```
def generate_run(params, grid_point):

    run_dir = prepare_run_dir(params, grid_point.model)

    write_run_files(
        params,
        grid_point.model,
        grid_point.evolution,
        run_dir,
    )
```

I tested this new setup with a general grid that I explore in section j.

f.3 New MesaGrid class

Since the new grids are a lot more complicated, the old MesaGrid class that I was could not understand the structure of the new class and I needed something more general.

The new class is still a bit barren and I am planning to expand it based on what I need. For now it has the following member functions and data:

```
class MesaGrid:
    def __init__(self, grid_dir):

        self.grid_dir = Path(grid_dir)

        with open(f"{grid_dir }/grid_settings.json") as f:
            self.settings = json.load(f)

        self.axes = {
            "R": self.settings["Rs"],
            "q": self.settings["qs"],
            "beta":
                self.settings["mass_transfer"]["beta"],
            "delta":
                self.settings["mass_transfer"]["delta"],
            "eps": self.settings["eps"],
        }

        self._load_models()

    def _load_models(self):
        self.models = []
        ...

    def filter(self, **filters):

        for run in self.models:

            keep = True

            for key, values in filters.items():

                if not isinstance(values, (list, tuple,
                    set)):
                    values = {values}

                if run.params[key] not in values:
                    keep = False
                    break

            if keep:
                yield run

    def array(self, value, x, y, **filters):

        xs = self.axes[x]
        ys = self.axes[y]

        arr = np.full((len(xs), len(ys)), np.nan)

        x_index = {np.round(v, 3): i for i, v in
            enumerate(xs)}
        print(x_index)
        y_index = {np.round(v, 3): i for i, v in
            enumerate(ys)}

        for run in self.filter(**filters):

            if callable(value):
                z = value(run)
            else:
                z = getattr(run, value)

            if x in ["beta", "delta"]:
                i = x_index[np.round(run.params[x] / (1 -
                    run.params["eps"]), 3)]
            else:
                i = x_index[np.round(run.params[x], 3)]

            if y in ["beta", "delta"]:
                j = y_index[np.round(run.params[y] / (1 -
                    run.params["eps"]), 3)]
            else:
                j = y_index[np.round(run.params[y], 3)]

            arr[i, j] = z

        return np.asarray(xs), np.asarray(ys), arr

class MesaRun:

    def __init__(self, run_dir):
        self.run_dir = run_dir

        with open(run_dir / "settings.json") as f:
            self.params = json.load(f)

        self.get_history()

        self.env_mass = self.history.envelope_mass
        self.q = self.history.star_2_mass /
            self.history.star_1_mass
        self.age = self.params["starting_age"] +
            self.star_age

    def __getattr__(self, name):
        return getattr(self.history, name)

    def get_history(self):
        try:
            with open(f"{self.run_dir}/history.pkl",
                "rb") as f:
                self.history = pickle.load(f)
        except:
            print(f"reading
                {self.run_dir}/LOGS/TPAGB/history.data")
            self.history =
                mr.MesaData(f"{self.run_dir}/LOGS/TPAGB/history.data")
            with open(f"{self.run_dir}/history.pkl",
                "wb") as f:
                pickle.dump(self.history, f,
                    protocol=pickle.HIGHEST_PROTOCOL)

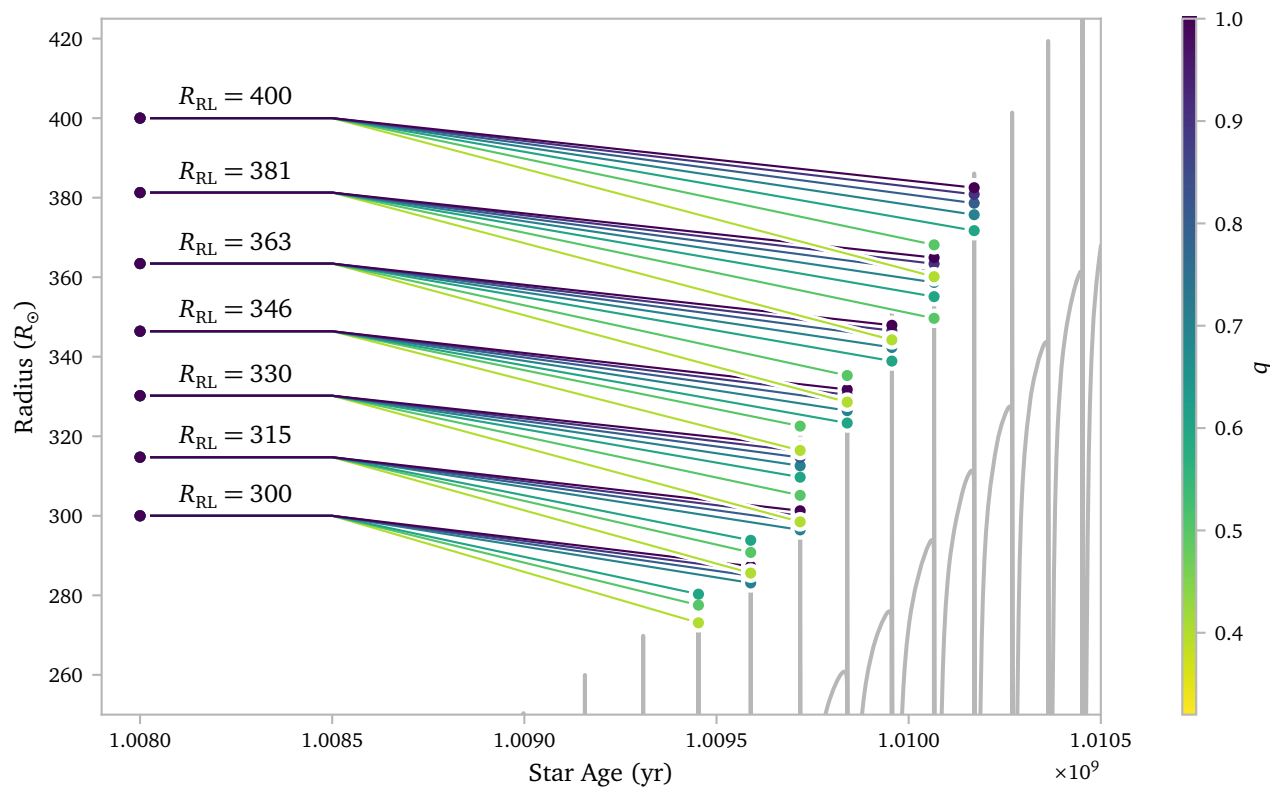
    def get_profiles(self):

        try:
            with open(f"{self.run_dir}/profiles.pkl",
                "rb") as f:
                self.profiles = pickle.load(f)

        except:
            profiles_dict =
                mr.MesaLogDir(f"{self.run_dir}/LOGS/TPAGB/")
            for i, profile in
                enumerate(profiles_dict.profile_numbers):
                    =
                        profiles_dict.profile_data(profile_number=profile)
            self.profiles =
                list(profiles_dict.profile_dict.values())
            with open(f"{self.run_dir}/profiles.pkl",
                "wb") as f:
                pickle.dump(self.profiles, f,
                    protocol=pickle.HIGHEST_PROTOCOL)
```

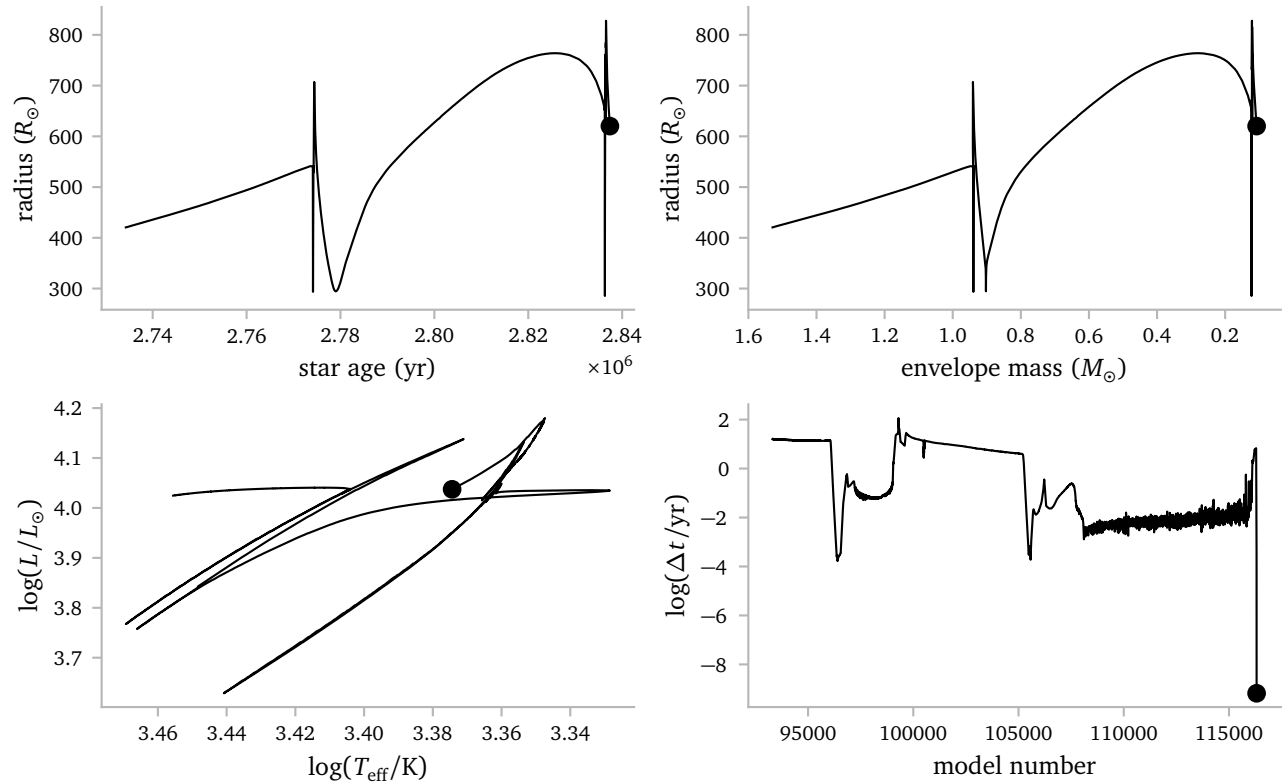
Here, I also added a MesaRun class to neatly combine the mesa history data as well as the parameters of the run.

g. Spacing



h. Debugging!

The $2.4 M_{\odot}$ star was behaving *REALLY* badly, and would not at all continue its evolution, despite all sorts of attempts to help the solver. Because of this, I decided to look into *what* exactly is going wrong. Let me start by plotting where the problem starts occurring.



It seems like the star cannot continue evolving just after its last thermal pulse. The radius and HR diagram look fine, but we see that the timestep goes to 0.

The terminal output shows me the usual *ÆSOPUS* warning, as well as the eos warning. I need to dig a bit deeper to figure out what is actually going wrong.

To start, I created a model just before the point where the star crashes. I also use the general debug options available to me in the inlists.

To start, I see this:

```

      solver_call_number      1
      gold tolerances level   1
      correction tolerances: norm, max 3.000000000000001D-05 3.000000000000001D-03
toll residual tolerances: norm, max 9.999999999999994D-12 1.000000000000001D-09
1 1 coeff 1.0000 avg resid 0.691E-05 max resid dv_dt 1 0.83380E+00 mix type xx000
      avg corr 0.206E-03 max corr lnd 1986 0.53235E-02 mix type 00000
      avg+max corr+resid
1 2 coeff 1.0000 avg resid 0.683E-05 max resid dv_dt 1 0.82426E+00 mix type xx000
      avg corr 0.573E-04 max corr lnd 2 0.12612E-01 mix type x0000
      avg+max corr+resid
do_eos_for_cell: get_eos ierr -1
do_eos_for_cell: get_eos ierr -1
do_eos_for_cell: get_eos ierr -1
do_eos_for_cell: get_eos ierr -1
do_eos_for_cell: get_eos ierr -1
do_eos_for_cell: get_eos ierr -1
do_eos_for_cell: get_eos ierr -1
do_eos_for_cell: get_eos ierr -1
do_eos_for_cell: get_eos ierr -1
do_eos_for_cell: get_eos ierr -1
do_eos_for_cell: get_eos ierr -1
do_eos returned ierr -1
set_hydro_vars failed in call to set_micro_vars
      set_hydro_vars returned ierr -1
      eval_equations: set_solver_vars returned ierr -1
adjust_correction: eval_equations returned ierr -1 1.0D+00 1.0D+00
```

The first error occurs in the `do_eos_for_cell` call!

I then use the GNU Project Debugger `gdb` to debug *MESA*. I write:

```
> OMP_NUM_THREADS=1 gdb ./star
(gdb) b eos_support.f90:89
(gdb) condition 1 logRho .lt. -25
```

The first line activates the debugger with the star executable where `OMP_NUM_THREADS=1` makes sure we are not multithreading. The second line creates a breakpoint at line 89 from the file `eos_support.f90`. Line 89 is the condition in `get_eos` that returns the error -1. The third line tell the breakpoint to only break if `logRho < -25`, which is the exact condition that returns the error.

This is the (relevant) output.

```

      solver_call_number      1
      gold tolerances level   1
      correction tolerances: norm, max 3.000000000000001D-05 3.000000000000001D-03
toll residual tolerances: norm, max 9.999999999999994D-12 1.000000000000001D-09
1 1 coeff 1.0000 avg resid 0.691E-05 max resid dv_dt 1 0.83380E+00 mix type xx000
      avg corr 0.206E-03 max corr lnd 1986 0.53235E-02 mix type 00000
      avg+max corr+resid
1 2 coeff 1.0000 avg resid 0.683E-05 max resid dv_dt 1 0.82426E+00 mix type xx000
      avg corr 0.573E-04 max corr lnd 2 0.12612E-01 mix type x0000
      avg+max corr+resid

Breakpoint 1, eos_support::get_eos (s=0xe7c0e0 <_star_data_def_MOD_star_handles>, k=1,
      xa=..., rho=<optimized out>, logrho=-40.561141182088775,
      t=228.31745818308877, logt=2.3585391208756734,
      res=..., dres_dlnrho=..., dres_dlnT=..., dres_dxa=...,
      ierr=0) at ../private/eos_support.f90:89

89      if(logRho < -25) then
```

This is the relevant source code:

```
59 ! $MESA_DIR/star/private/eos_support.f90
60
61 subroutine get_eos( &
62     s, k, xa, &
63     Rho, logRho, T, logT, &
64     res, dres_dlnRho, dres_dlnT, &
65     dres_dxa, ierr)
66
67     use eos_lib, only: eosDT_get
68     use eos_def, only: num_eos_basic_results,
69         num_eos_d_dxa_results, num_helm_results,
70         i_lne
71
72     type (star_info), pointer :: s
73     integer, intent(in) :: k ! 0 means not being
74         called for a particular cell
75     real(dp), intent(in) :: xa(:), Rho, logRho, T,
76         logT
77     real(dp), dimension(num_eos_basic_results),
78         intent(out) :: &
79         res, dres_dlnRho, dres_dlnT
80     real(dp), intent(out) ::
81         dres_dxa(num_eos_d_dxa_results,s% species)
82     integer, intent(out) :: ierr
83
84     real(dp), dimension(num_eos_basic_results) ::
85         dres_dabar, dres_dzbar
86     integer :: j
87     logical :: off_table
88
89     include 'formats'
90
91     ierr = 0
92
93     if (s% doing_timing) &
94         s% timing_num_get_eos_calls = s%
95             timing_num_get_eos_calls + 1
96
97     if(logRho < -25) then
98         ! Provide some hard lower limit on what we
99         ! would even try to evaluate the eos at
100        ! Going to low causes FPE's when we try to
101        ! evaluate certain derviatives that need
102        ! (rho**power)
103        s% retry_message = 'eos evaluted at too low a
104            density'
105        ierr = -1
106        return
107    end if
108    ...
```

Clearly, something is going wrong with the *density*! In this case, the first zone ($k=1$, at the surface) has a density of $\log(\rho) \approx -40$. Obviously this is insane. The temperature is very low as well and wrong.

So, really we need to look back a little bit, not at `get_eos` where the *error* originates, but a little before that, where the back values originate.

Below, I show the source code that calls `get_eos`:

```

310 ! $MESA_DIR/star/private/micro.f90
311
312 subroutine do_eos_for_cell(s,k,ierr)
313
314     use const_def
315     use chem_def
316     use chem_lib
317     use eos_def
318     use eos_lib, only: eval_PC_fraction
319     use eos_support
320     use net_def,only:net_general_info
321     use star_utils, only: write_eos_call_info
322
323     type (star_info), pointer :: s
324     integer, intent(in) :: k
325     integer, intent(out) :: ierr
326
327     real(dp), dimension(num_eos_basic_results) :: &
328         res, res_a, res_b, d_dln, d_dlnT, d_eos_dabar, d_eos_dzbar
329     real(dp) :: &
330         sumx, dx, dxh_a, dxh_b, &
331         Rho, logRho, lnd, lnE, logT, T, energy, logQ, frac
332     integer, pointer :: net_iso(:)
333     integer :: j, species
334     real(dp), parameter :: epsder = 1d-4, Z_limit = 0.5d0
335
336     logical, parameter :: testing = .false.
337
338     include 'formats'
339
340     ierr = 0
341
342     net_iso => s% net_iso
343     species = s% species
344     call basic_composition_info( &
345         species, s% chem_id, s% xa(1:species,k), s% X(k), s% Y(k), s% Z(k), &
346         s% abar(k), s% zbar(k), s% z2bar(k), s% ye(k), s% mass_correction(k), sumx)
347
348     logT = s% lnT(k)/ln10
349
350     if (s% lnPgas_flag) then
351         if (s% Pgas(k) == 0) then
352             ierr = -1
353             return
354             write(*,2) 's% lnPgas(k)/ln10', k, s% lnPgas(k)/ln10
355             stop 'do_eos_for_cell s% Pgas(k) == 0'
356         end if
357         call get_peos( &
358             s, k, s% Z(k), s% X(k), s% abar(k), s% zbar(k), s% xa(:,k), &
359             s% Pgas(k), s% lnPgas(k)/ln10, s% T(k), logT, &
360             Rho, logRho, s% dlnRho_dlnPgas_const_T(k), s% dlnRho_dlnT_const_Pgas(k), &
361             res, s% d_eos_dln(:,k), s% d_eos_dlnT(:,k), &
362             d_eos_dabar, d_eos_dzbar, ierr)
363         s% lnd(k) = logRho*ln10
364         s% rho(k) = Rho
365         call store_stuff(ierr)
366         return
367     end if
368
369     logRho = s% lnd(k)/ln10
370
371     call get_eos( &
372         s, k, s% Z(k), s% X(k), s% abar(k), s% zbar(k), s% xa(:,k), &
373         s% rho(k), logRho, s% T(k), logT, &
374         res, s% d_eos_dln(:,k), s% d_eos_dlnT(:,k), &
375         d_eos_dabar, d_eos_dzbar, &
376         ierr)
377     if (ierr /= 0) then
378         if (s% report_ierr) then
379             write(*,*) 'do_eos_for_cell: get_eos ierr', ierr
380             stop 'do_eos_for_cell'
381         end if
382         return
383     end if
384     s% lnPgas(k) = res(i_lnPgas)
385     s% Pgas(k) = exp_cr(s% lnPgas(k))
386     ...

```

The errors occur in line 344. We can also look at the backtrace of the get_eos call:

```

(gdb) bt
#0 eos_support::get_eos (s=0xe7c0e0
↳ __star_data_def_MOD_star_handles>, k=1, xa=...,
↳ rho=<optimized out>, logrho=-40.561141182088775,
↳ t=228.31745818308877, logt=2.3585391208756734,
↳ res=..., dres_dlnrho=..., dres_dlnT=...,
↳ dres_dxa=..., ierr=0) at
↳ ../private/eos_support.f90:89
#1 0x000000000503d11 in micro::do_eos_for_cell
↳ (s=<optimized out>, k=<optimized out>, ierr=0) at
↳ ../private/micro.f90:385
#2 0x0000000004422ec in
↳ star_utils::_star_utils_MOD_foreach_cell_omp_fn.0
↳ () at ../private/star_utils.f90:72
#3 0x00007fea9e211f42 in GOMP_parallel (fn=0x442260
↳ <star_utils::_star_utils_MOD_foreach_cell_omp_fn.0>,
↳ data=0x7ffcbdc100b0, num_threads=1, flags=0)
↳ at
↳ /home/user/sdk2-tmp/build/gcc/libgomp/parallel.c:171
#4 0x000000000456935 in star_utils::foreach_cell
↳ (s=0xe7c0e0 __star_data_def_MOD_star_handles>,
↳ nzlo=<optimized out>, nzhi=<optimized out>,
↳ use_omp=<optimized out>,
↳ dol=0x5036c0 <micro::do_eos_for_cell>, ierr=0) at
↳ ../private/star_utils.f90:72
#5 0x0000000005057c3 in micro::do_eos (ierr=0,
↳ nzhi=<optimized out>, nzlo=<optimized out>,
↳ s=<optimized out>) at ../private/micro.f90:271
#6 micro::set_eos (ierr=0) at ../private/micro.f90:217
#7 micro::set_micro_vars (nzlo=1, nzhi=5857,
↳ skip_eos=.FALSE., skip_net=.FALSE., skip_neu=.FALSE.,
↳ skip_kap=.FALSE., ierr=0) at ../private/micro.f90:97
#8 0x00000000050cbe7 in hydro_vars::set_hydro_vars
↳ (s=0xe7c0e0 __star_data_def_MOD_star_handles>,
↳ nzlo=1, nzhi=5857, skip_basic_vars=.TRUE.,
↳ skip_micro_vars=.FALSE.,
↳ skip_m_grav_and_grav=.TRUE., skip_eos=.FALSE.,
↳ skip_net=.FALSE., skip_neu=.FALSE.,
↳ skip_kap=.FALSE., skip_grads=.TRUE.,
↳ skip_rotation=.TRUE., skip_brunt=.TRUE.,
↳ skip_other_cgrav=.TRUE., skip_mixing_info=.TRUE.,
↳ skip_set_cz_bdy_mass=.TRUE., skip_mlt=.FALSE.,
↳ ierr=0) at ../private/hydro_vars.f90:506
#9 0x0000000007fa3fd in
↳ solver_support::set_vars_for_solver (s=0xe7c0e0
↳ __star_data_def_MOD_star_handles>, nvar=13, nzlo=1,
↳ nzhi=<optimized out>, dt=69894018.156770468, ierr=0)
↳ at ../private/solver_support.f90:1107
#10 0x0000000007fe7a9 in solver_support::set_solver_vars
↳ (ierr=0, dt=69894018.156770468, nvar=13, s=0xe7c0e0
↳ __star_data_def_MOD_star_handles>) at
↳ ../private/solver_support.f90:815
#11 solver_support::eval_equations (s=0xe7c0e0
↳ __star_data_def_MOD_star_handles>, nvar=13, ierr=0)
↳ at ../private/solver_support.f90:113
#12 0x0000000007bd036 in do_solver::do_equations
↳ (ierr=0) at ../private/star_solver.f90:736
#13 0x0000000007c2960 in star_solver::adjust_correction
↳ (_err_msg=256, _err_msg=256, ierr=0, err_msg=<error
↳ reading variable: Cannot access memory at address
↳ 0x9>, coeff=1, slope=0, f=0,
↳ grad_f=<error reading variable: value requires 609128
↳ bytes, which is more than max-value-size>,
↳ max_corr_coeff=1, min_corr_coeff_in=<optimized
↳ out>) at ../private/star_solver.f90:848
#14 star_solver::do_solver (s=0xe7c0e0
↳ __star_data_def_MOD_star_handles>, nvar=13, afl=...,
↳ ldaf=39, neq=76141,
↳ skip_global_corr_coeff_limit=.TRUE.,
↳ gold_tolerances_level=1,
↳ tol_max_correction=0.0030000000000000001,
↳ tol_correction_norm=3.000000000000001e-05,
↳ work=<error reading variable: value requires
↳ 61521928 bytes, which is more than
↳ max-value-size>,
↳ lwork=7690241, iwork=<error reading variable: value
↳ requires 304564 bytes, which is more than
↳ max-value-size>, liwork=76141,
↳ convergence_failure=.FALSE., ierr=0)
↳ at ../private/star_solver.f90:317
#15 0x0000000007c5823 in star_solver::solver (s=0xe7c0e0
↳ __star_data_def_MOD_star_handles>, nvar=13,
↳ skip_global_corr_coeff_limit=.TRUE.,
↳ gold_tolerances_level=1,
↳ tol_max_correction=0.0030000000000000001,
↳ tol_correction_norm=3.000000000000001e-05,
↳ work=<error reading variable: value requires
↳ 61521928 bytes, which is more than
↳ max-value-size>,
↳ lwork=7690241, iwork=<error reading variable: value
↳ requires 304564 bytes, which is more than
↳ max-value-size>, liwork=76141,
↳ convergence_failure=.FALSE., ierr=0)
↳ at ../private/star_solver.f90:104

```

I think the interesting calls are #8 and #13.

- Function call 8 sets the hydro_vars, which later give the error.
- Function call 13 adjusts the corrections after the previous newton iteration was unable to converge.

To continue debugging, I need to figure out *where* the wrong density values come from.

To do this, I use grep to look for all occurrences of density assignments:

```

> grep -R "s% *lnd.*=" star/private
star/private/predictive_mix.f90:      s% lnd(k) = lnd_save(k)
star/private/hydro_rsp2_support.f90:  s% lnd(k) = log(s% rho(k))
star/private/micro.f90:               s% lnd(k) = logRho*ln10
star/private/star_utils.f90:         s% lnd_start(k) = -ld99
star/private/rsp_eval_eos_and_kap.f90: s% lnd(k) = logRho*ln10
star/private/rsp_eval_eos_and_kap.f90: s% lnd(k) = logRho*ln10
star/private/rsp_eval_eos_and_kap.f90: s% lnd(kk) = logRho_result*ln10
star/private/conv_premix.f90:         s%lnd(kc_t:kc_b) = sd%lnd(kc_t:kc_b)
star/private/hydro_vars.f90:         s% lnd(k) = s% xh(i_lnd,k)
star/private/hydro_vars.f90:         s% lnd_start(k) = s% lnd(k)
star/private/solver_support.f90:       s% lnd(k) = x(i_lnd)
star/private/struct_burn_mix.f90:      !s% lnd_start(k) = s% lnd(k) set elsewhere
star/private/write_model.f90:         call writel(s% lnd(k),ierr); if (ierr /= 0) exit

```

Really, there are only a two lines here, namely:

```

star/private/hydro_vars.f90:         s% lnd(k) = s% xh(i_lnd,k)
star/private/solver_support.f90:       s% lnd(k) = x(i_lnd)

```

We can again run the debugger and set breakpoints at these lines:

```
(gdb) b hydro_vars.f90:299
Breakpoint 1 at 0x50dbb2: file ../private/hydro_vars.f90, line 299.
(gdb) b solver_support.f90:1337
Breakpoint 2 at 0x7f7194: file ../private/solver_support.f90, line 1337.
(gdb) condition 2 x(i_lnd) < -30
(gdb) run
```

Eventually, we reach the state:

```

               solver_call_number      1
               gold tolerances level   1
               correction tolerances: norm, max  3.0000000000000001D-05  3.0000000000000001D-03
tol1 residual tolerances: norm, max  9.9999999999999994D-12  1.0000000000000001D-09
1 1 1  coeff 1.0000 avg resid  0.691E-05      max resid dv_dt      1  0.83380E+00 mix type xx000
               avg corr  0.206E-03      max corr lnd  1986  0.53235E-02 mix type 00000
               avg+max corr+resid
1 2 1  coeff 1.0000 avg resid  0.683E-05      max resid dv_dt      1  0.82426E+00 mix type xx000
               avg corr  0.573E-04      max corr lnd      2  0.12612E-01 mix type x0000
               avg+max corr+resid
```

```
Breakpoint 2, set_vars_for_solver::set1 (k=1, report=.FALSE., ierr=0)
  at ../private/solver_support.f90:1337
1337      s% lnd(k) = x(i_lnd)
(gdb) p x(i_lnd)
$1 = -93.395479040704487
(gdb) p s% solver_dx(i_lnd,k)
$2 = -67.602505267054823
```

This is the first place where the density becomes very wrong. And in fact, $\ln \rho = -93.39 \approx \log \rho = -40.56$. We see that at this point, p $s\%$ $\text{solver_dx}(i_lnd,k) = -67.60 \dots$, which is a really extreme correction.

Why is this?

To figure this out, we need to dive deeper into the code responsible for the corrections.

```
> grep -R "s%.*solver_dx,.*=" star/private
star/private/star_solver.f90:      s% solver_dx(i,k) = dxsave(i,k)
star/private/star_solver.f90:      s% solver_dx(i,k) = s% solver_dx(i,k) + s% x_scale(i,k)*soln(i,k)
star/private/star_solver.f90:      s% solver_dx(i,k) = s% solver_dx(i,k) + coeff*s% x_scale(i,k)*soln(i,k)
star/private/star_solver.f90:      s% solver_dx(i,k) = dxsave(i,k) + s% x_scale(i,k)*soln(i,k)
star/private/star_solver.f90:      s% solver_dx(i,k) = dxsave(i,k) + coeff*s% x_scale(i,k)*soln(i,k)
star/private/star_solver.f90:      s% solver_dx(i_var,k+k_off) = save_dx(i_var,k+k_off) + delta_x
star/private/star_solver.f90:      s% solver_dx(i_var_sink,k+k_off) = save_dx(i_var_sink,k+k_off) - delta_x
star/private/star_solver.f90:      s% solver_dx(i_var,k+k_off) = save_dx(i_var,k+k_off)
star/private/star_solver.f90:      s% solver_dx(i_var_sink,k+k_off) = save_dx(i_var_sink,k+k_off)
star/private/alloc.f90:      s% solver_dx(1:nvar,1:nz) => s% solver_dx1(1:nvar*nz)
star/private/alloc.f90:      s% solver_dx(1:nvar,1:nz) => s% solver_dx1(1:nvar*nz)
star/private/struct_burn_mix.f90: s% solver_dx(j1,k) = s% xh(j1,k) - s% xh_start(j1,k)
star/private/struct_burn_mix.f90: s% solver_dx(j1,k) = s% xa_sub_xa_start(j2,k)
```

I think the easiest way to proceed is to actually *watch* $s\%$ $\text{solver_dx}(i,k)$. In particular, we want to watch $i = 1$ and $k = 1$, where k is the zone, and $i = 1$ corresponds to the $\ln \rho$.

To set this up, we need to make sure we can access the right object, so we have to take some preparation steps. This is the full list of commands.

```
> OMP_NUM_THREADS=1 /usr/bin/gdb ./star
(gdb) b do_solver
(gdb) run
(gdb) c
Continuing.
```

```
Breakpoint 1.2, star_solver::do_solver (s=0xe7c0e0
↳ <__star_data_def_MOD_star_handles>, nvar=13,
↳ afl=<error reading variable: value requires 23821592
↳ bytes, which is more than max-value-size>,
  ldaf=39, neq=76141,
  ↳ skip_global_corr_coeff_limit=.TRUE.,
  ↳ gold_tolerances_level=1,
  ↳ tol_max_correction=0.0030000000000000001,
  ↳ tol_correction_norm=3.0000000000000001e-05,
work=<error reading variable: value requires 61521928
↳ bytes, which is more than max-value-size>,
  ↳ lwork=7690241,
iwork=<error reading variable: value requires 304564
↳ bytes, which is more than max-value-size>,
  ↳ liwork=76141, convergence_failure=.TRUE.,, ierr=0)
↳ at ../private/star_solver.f90:109
109      subroutine do_solver( &
(gdb) delete 1
(gdb) set $star = s
(gdb) watch $star% solver_dx(1,1)
(gdb) b star_solver.f90:848
```

Okay! Let's look at the results:

```
Hardware watchpoint 2: $star% solver_dx(1,1)

Old value = 0
New value = 0.012397313114644006
apply_coeff (just_use_dx=<optimized out>, coeff=1,
↳ soln=<error reading variable: value requires 609128
↳ bytes, which is more than max-value-size>,
  dxsave=<error reading variable: value requires 609128
  ↳ bytes, which is more than max-value-size>,
  ↳ nz=5857, nvar=<optimized out>) at
  ↳ ../private/star_solver.f90:996
996      do i=1,nvar
(gdb) c
Continuing.
```

```
Breakpoint 3, adjust_correction (_err_msg=256,
↳ _err_msg=256, ierr=0, err_msg=..., coeff=1, slope=0,
↳ f=0,
  grad_f=<error reading variable: value requires 609128
  ↳ bytes, which is more than max-value-size>,
  ↳ max_corr_coeff=1, min_corr_coeff_in=<optimized
  ↳ out>) at ../private/star_solver.f90:848
848      call do_equations(ierr)
(gdb) p iter
$1 = 1
(gdb) c
Continuing.
1 1 1  coeff 1.0000 avg resid  0.691E-05      max
↳ resid dv_dt      1  0.83380E+00 mix type xx000
↳ avg corr  0.206E-03      max corr lnd  1986
↳ 0.53235E-02 mix type 00000  avg+max corr+resid
```

```
Hardware watchpoint 2: $star% solver_dx(1,1)

Old value = 0.012397313114644006
New value = 0.33770784939148324
apply_coeff (just_use_dx=<optimized out>, coeff=1,
↳ soln=<error reading variable: value requires 609128
↳ bytes, which is more than max-value-size>,
  dxsave=<error reading variable: value requires 609128
  ↳ bytes, which is more than max-value-size>,
  ↳ nz=5857, nvar=<optimized out>) at
  ↳ ../private/star_solver.f90:996
996      do i=1,nvar
(gdb) c
Continuing.
```

```
Breakpoint 3, adjust_correction (_err_msg=256,
↳ _err_msg=256, ierr=0, err_msg=..., coeff=1, slope=0,
↳ f=0,
  grad_f=<error reading variable: value requires 609128
  ↳ bytes, which is more than max-value-size>,
  ↳ max_corr_coeff=1, min_corr_coeff_in=<optimized
  ↳ out>) at ../private/star_solver.f90:848
848      call do_equations(ierr)
(gdb) p iter
$2 = 1
(gdb) c
Continuing.
1 2 1  coeff 1.0000 avg resid  0.683E-05      max
↳ resid dv_dt      1  0.82426E+00 mix type xx000
↳ avg corr  0.573E-04      max corr lnd      2
↳ 0.12612E-01 mix type x0000  avg+max corr+resid
```

```
Hardware watchpoint 2: $star% solver_dx(1,1)

Old value = 0.33770784939148324
New value = -67.602505267054823
apply_coeff (just_use_dx=<optimized out>, coeff=1,
↳ soln=<error reading variable: value requires 609128
↳ bytes, which is more than max-value-size>,
  dxsave=<error reading variable: value requires 609128
  ↳ bytes, which is more than max-value-size>,
  ↳ nz=5857, nvar=<optimized out>) at
  ↳ ../private/star_solver.f90:996
996      do i=1,nvar
(gdb) c
Continuing.
```

```
Breakpoint 3, adjust_correction (_err_msg=256,
↳ _err_msg=256, ierr=0, err_msg=..., coeff=1, slope=0,
↳ f=0,
  grad_f=<error reading variable: value requires 609128
  ↳ bytes, which is more than max-value-size>,
  ↳ max_corr_coeff=1, min_corr_coeff_in=<optimized
  ↳ out>) at ../private/star_solver.f90:848
848      call do_equations(ierr)
(gdb) p iter
$3 = 1
(gdb)
```

Clearly, we are getting a wrong $s\%$ $\text{solver_dx}(1,1)$ from apply_coeff . It gets changed like this:

```
s% solver_dx(i,k) = s% solver_dx(i,k) + s%
↳ x_scale(i,k)*soln(i,k)
```

I inspected $s\%$ x_scale and soln throughout the Newton Iterations. The scale is consistent, while the solution at some point just becomes very large.

It also makes sense that we see the following in the terminal directly following the wrong density:

```
do_eos_for_cell: get_eos ierr      -1
do_eos returned ierr      -1
set_hydro_vars failed in call to set_micro_vars
               set_hydro_vars returned ierr
               ↳ -1
               eval_equations: set_solver_vars returned ierr
               ↳ -1
```

This is because directly following the call to apply_coeff , the routine adjust_correction calls do_equations , which calls eval_equations

```
> set_solver_vals > set_vars_for_solver > set_hydro_vars >
set_micro_vars > set_eos > do_eos > foreach_cell > do_eos_for_cell
> get_eos.
```

This is the place where the first error occurs due to a density that is too small.

We probably want to look how we obtain the wrong $\text{soln}(1,1)$.

The scale is computed according to subroutine $\text{set_xscale_info}(s, nvar, ierr)$ in $\text{solver_support.f90}$.

Here, for normal variables¹, the scale is determined according to

```
s% x_scale(i,k) = max(1, abs(s% xh_start(i,k)))
```

The values we find for the scale align well with this equation, so it must be the solution to the Newton-Raphson solver that is wrong.

This scheme is represented by repeatedly trying to solve (Paxton et al., 2011)

$$0 = \vec{F}(\vec{y}) = \vec{F}(\vec{y}_i + \delta \vec{y}_i) = \vec{F}(\vec{y}_i) + \left[\frac{d\vec{F}}{d\vec{y}} \right]_i \delta \vec{y}_i + \mathcal{O}(\delta \vec{y}_i^2).$$

Here, y_i is a trial solution, $\vec{F}(\vec{y}_i)$ is the residual of the trial solution, $\delta \vec{y}_i$ is the correction to the trial solution and $[d\vec{F} / d\vec{y}]_i$ is the Jacobian matrix. The “solutions” from MESA are the $\delta \vec{y}_i$ ’s, which get scaled by the scale to obtain the changes to the star.

For a single iteration, MESA solves for $\delta \vec{y}$ in

$$J \delta \vec{y} = -\vec{F}(\vec{y}).$$

To make sure the Newton-Raphson solver is working correctly, I access the Jacobian, correction and the residual.

I then compute the LHS and the RHS of the equation above and test their agreement².

```
lhs = dblk_1.T @ delta_corr_1 + ublk_1.T @ delta_corr_2
rhs = -1 * equ
print(lhs - rhs)
```

```
[9.43255890e-18  1.06581410e-14  0.00000000e+00
 ↪  2.41035957e-15
 3.64629801e-63  1.21543267e-63  2.09119527e-85
 ↪  0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00
 ↪  0.00000000e+00
0.00000000e+00]
```

Nice. The Newton-Raphson solver works! the Problem must occur *before* the solver.

This also means the absurd correction of -2.634 is the *correct* value based on the residuals and the Jacobian.

Let’s see how the corrections are constructed, based on the individual components of the Jacobian and the residuals.

From now on, I will only consider the four hydro_vars. This means I am *excluding* the abundances, which do not seem to impact the first zone anyway.

The Jacobian for the self interaction of the first zone is defined like this:

$$\mathbf{J}_{1 \rightarrow 1} = \begin{bmatrix} \frac{\partial y_\rho}{\partial \ln \rho} & \frac{\partial y_\rho}{\partial \ln T} & \frac{\partial y_\rho}{\partial \ln R} & \frac{\partial y_\rho}{\partial \ln L} \\ \frac{\partial y_T}{\partial \ln \rho} & \frac{\partial y_T}{\partial \ln T} & \frac{\partial y_T}{\partial \ln R} & \frac{\partial y_T}{\partial \ln L} \\ \frac{\partial y_R}{\partial \ln \rho} & \frac{\partial y_R}{\partial \ln T} & \frac{\partial y_R}{\partial \ln R} & \frac{\partial y_R}{\partial \ln L} \\ \frac{\partial y_L}{\partial \ln \rho} & \frac{\partial y_L}{\partial \ln T} & \frac{\partial y_L}{\partial \ln R} & \frac{\partial y_L}{\partial \ln L} \end{bmatrix}_{1 \rightarrow 1}, \delta \vec{y}_1 = \begin{bmatrix} \delta y_\rho \\ \delta y_T \\ \delta y_R \\ \delta y_L \end{bmatrix}_1, \vec{F}(\vec{y}_1) = \begin{bmatrix} F(y_\rho) \\ F(y_T) \\ F(y_R) \\ F(y_L) \end{bmatrix}_1$$

Now, importantly, this self interaction is *not the only* part that influences the first zone. MESA has couplings to the next and previous cell, where the complete matrix is split in a block diagonal form like this:

$$\mathbf{A} = \begin{matrix} & \text{zone 1} & \text{zone 2} & \text{zone 3} & \text{zone 4} & \text{zone 5} \\ \begin{matrix} \text{zone 1} \\ \text{zone 2} \\ \text{zone 3} \\ \text{zone 4} \\ \text{zone 5} \end{matrix} & \begin{bmatrix} D_1 & U_1 & 0 & & \\ L_2 & D_2 & U_2 & 0 & \\ 0 & L_3 & D_3 & U_3 & 0 \\ & 0 & L_4 & D_4 & U_4 \\ & & 0 & L_5 & D_5 \end{bmatrix} \end{matrix}.$$

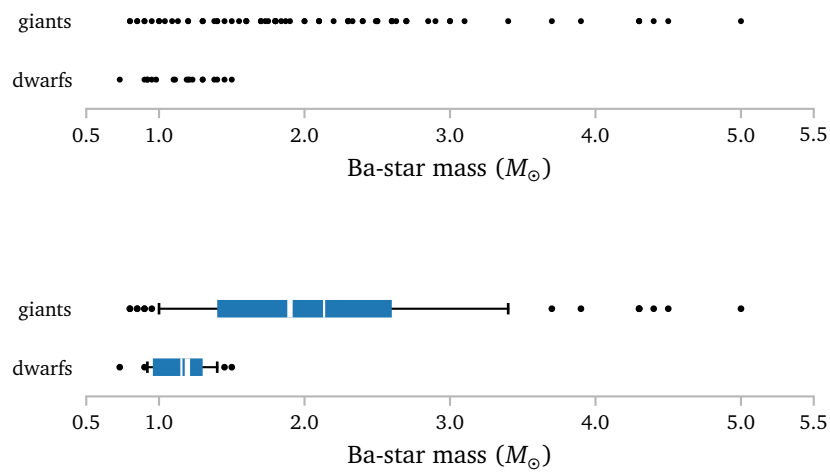
In the case of zone 1, only D_1 and U_1 matter³.

1 not rotation or abundances

2 for the first zone

3 For a more general zone k , we have L_k , D_k and U_k .

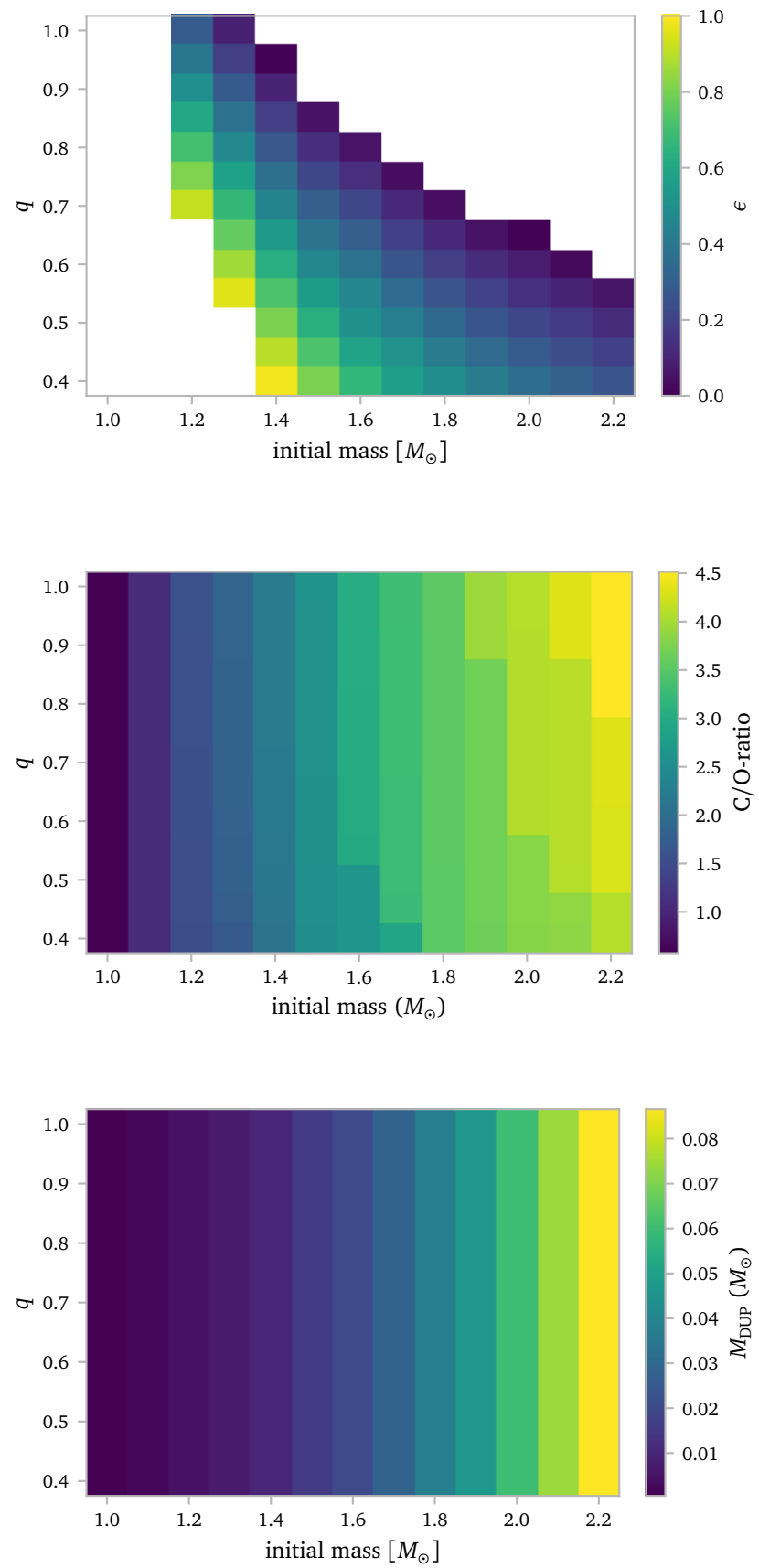
i. Ba-dwarf masses



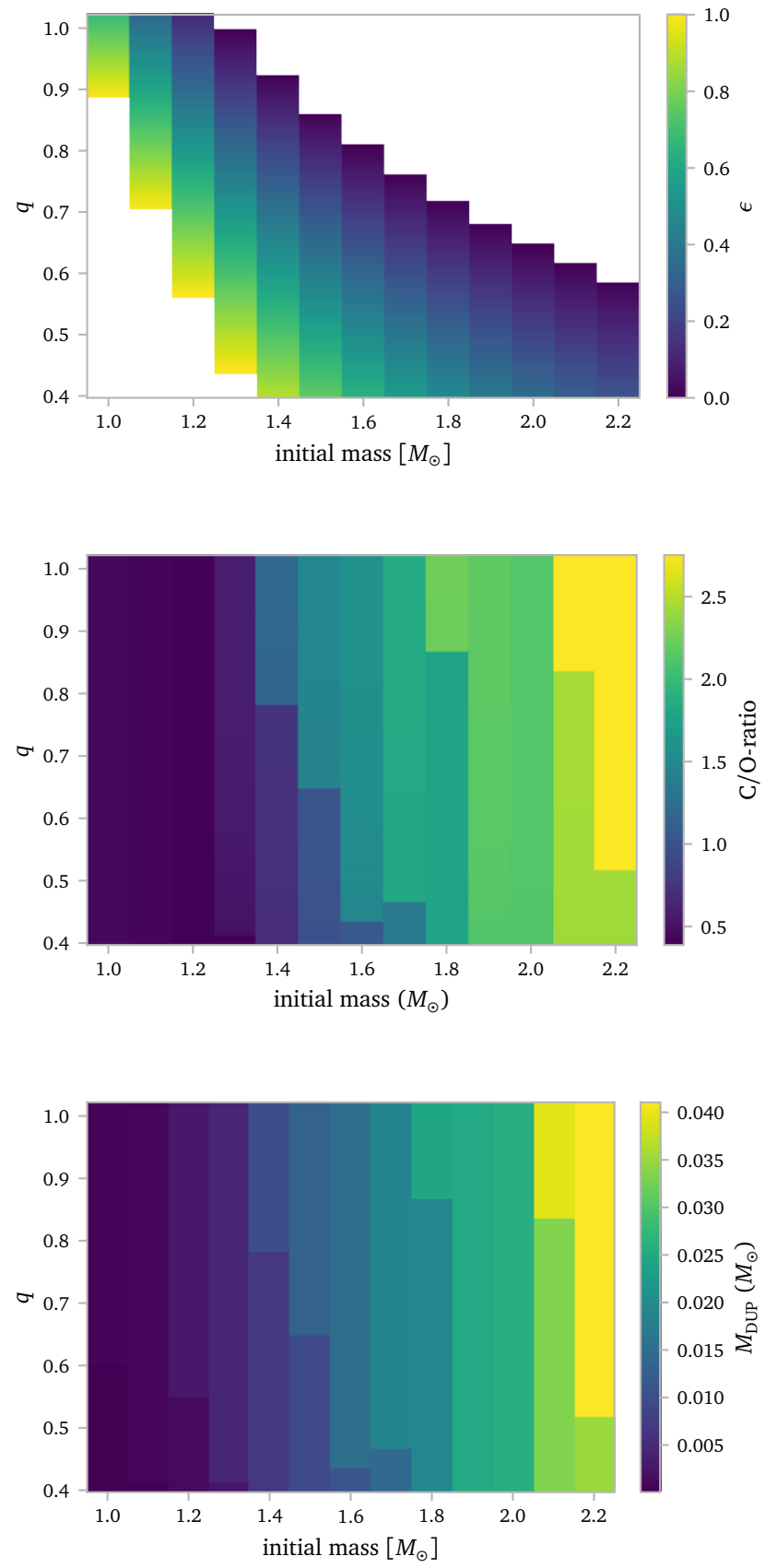
When considering the Barium star masses, the dwarfs are much more easily modelled by a single value around $M_{\text{Ba}} = 1.2 M_{\odot}$. This aligns well with the results from Sarmah et al. (2026), who find an average mass of $M_{\text{Ba}} = 1.29 \pm 0.09 M_{\odot}$.

From now on, I will use this value.

Let's say the binary interacts at a Roche lobe radius of $450 M_{\odot}$, we find:



Now, for a Roche lobe radius of $450 M_{\odot}$, we find:



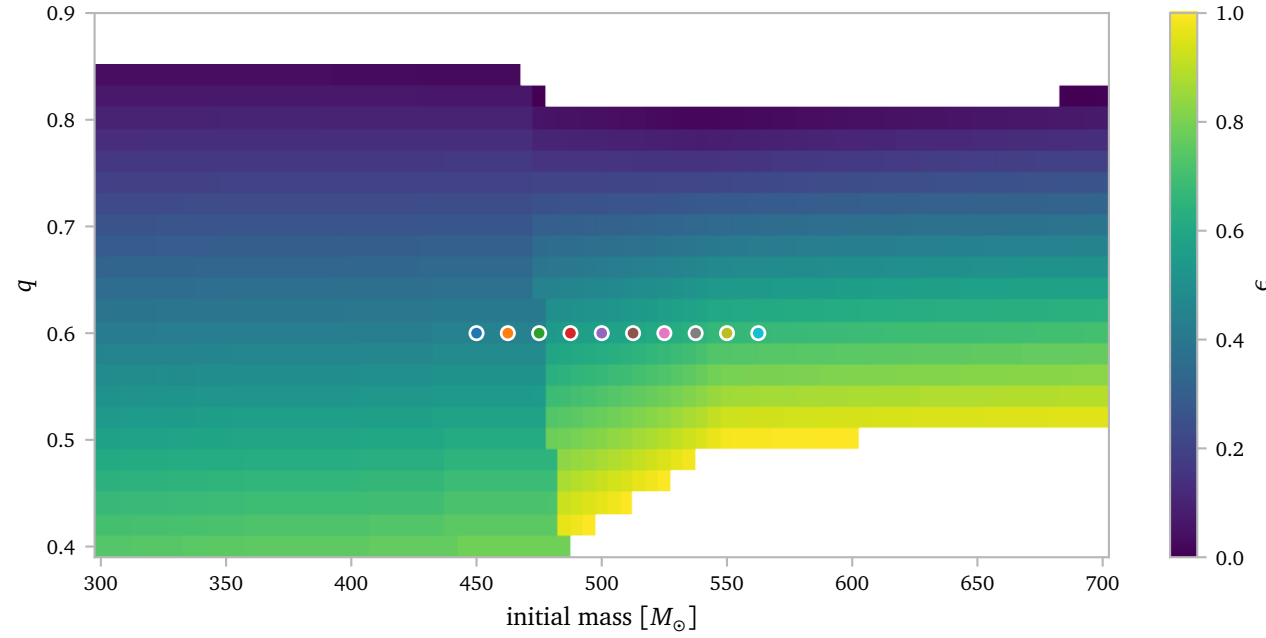
It is clear that the heavy stars have trouble producing the barium star masses.

Now, obviously, there might be positive correlation between the barium star mass and the TP-AGB mass, but it should still be obvious that only the lighter TP-AGB stars can accrete significant mass without starting with a very unequal mass ratio.

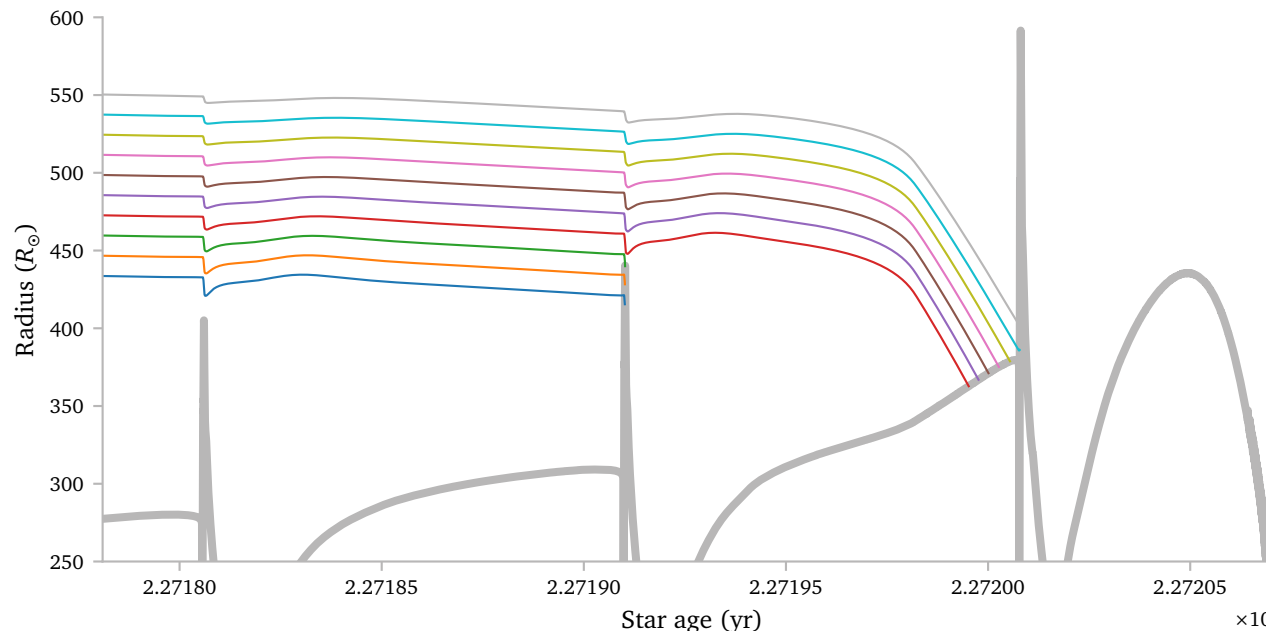
i.1 Changing interaction radius

Instead of varying the mass, we can also vary the interaction radius. This setup is lot more like my previous grids.

For the $M_i = 1.5 M_{\odot}$ model, we get the following plot:



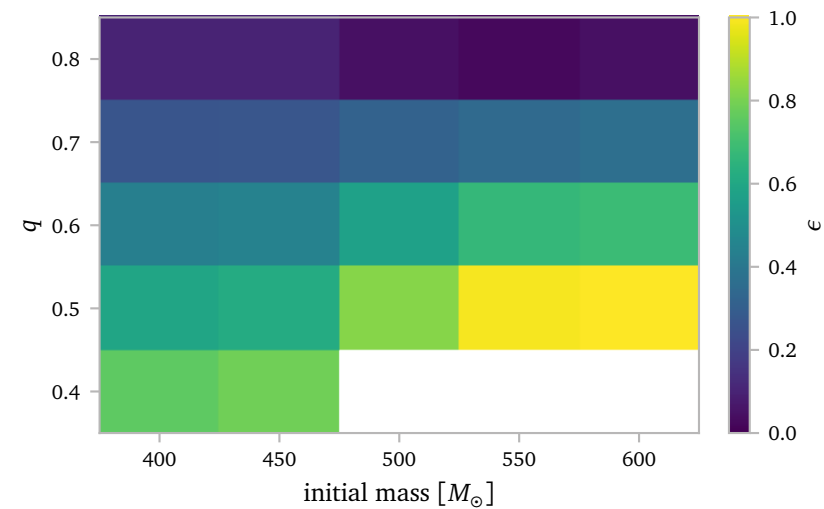
There are two visible boundaries here. The first one is very obvious and occurs at the point where the Vassiliadis & Wood (1993) winds start. The second boundary is a bit less obvious. It occurs when the models interact during the last pulse instead of during the interpulse before the last pulse. The figure below shows this for a constant q .



The colors of the lines correspond to the colors of the dots of the previous figure. It now also makes sense that the possible q -range shrinks for larger R . Clearly, here the star has already lost a lot of mass due to winds, which it can no longer accrete during RLOF. The winds also transfer some mass, which shifts the upper limit down a little bit⁴.

⁴ although a lot less than the lower limit shifts upwards.

We can use something like this to generate a grid as well:

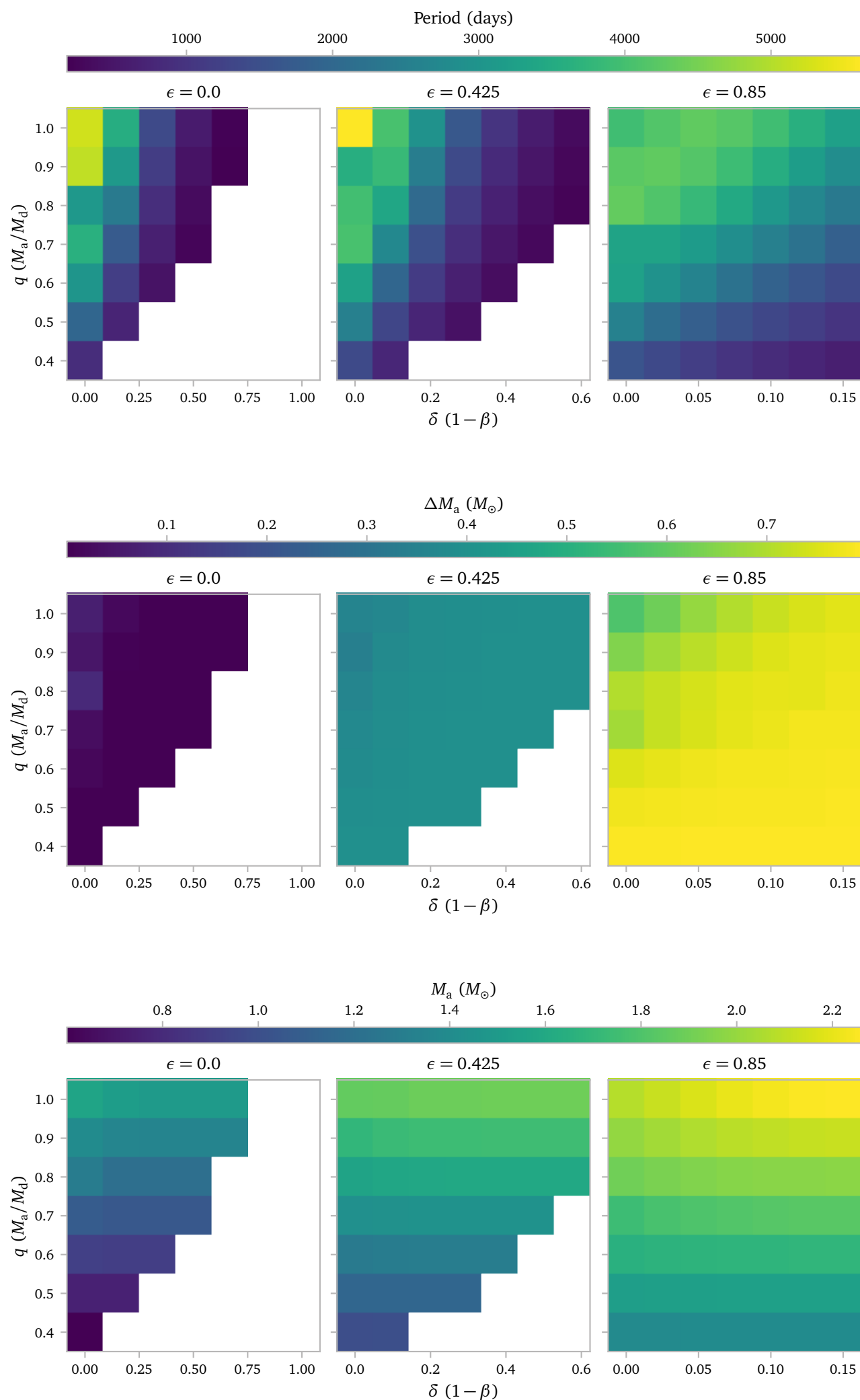


My idea is to use the ϵ value to ensure we produce a barium star with the appropriate mass. Once we have a value for ϵ , we can then vary β and δ to produce the required ϵ value.

j. Grid

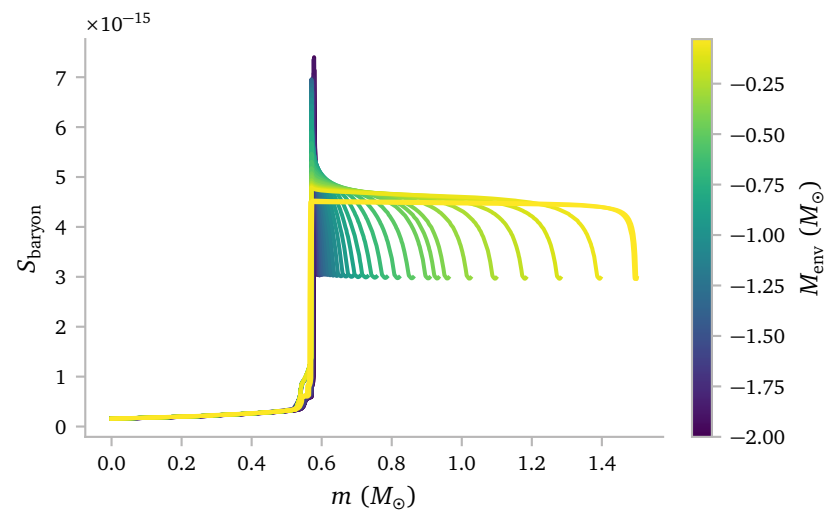
j.1 History

My grid has $R_{\text{RL}} = 350 R_{\odot}$ and three different epsilon values. For every epsilon value, I vary the initial mass ratio and the amount of mass that gets lost through isotropic re-emission and a CCT.



j.2 Profiles

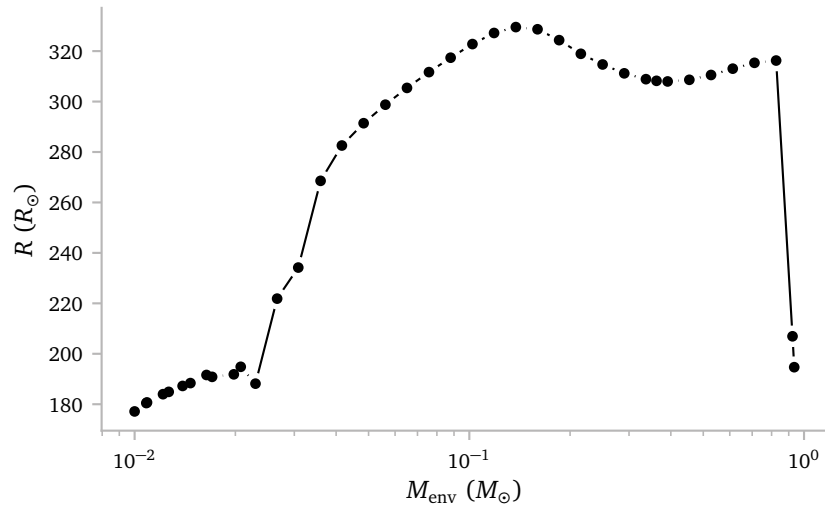
This section shows some tests for the MESA profiles. To start, here we can clearly see the profile saving code working.



The binary saves at most 50 profiles up to $M_{\text{env}} = 0.001 M_{\odot}$. The profiles are saved with a logarithmic spacing between the initial envelope mass and the final envelope mass. Most models will terminate before reaching $M_{\text{env}} = 0.001 M_{\odot}$ and will therefore have slightly fewer profiles. In this case, we have 38 models going down to $M_{\text{env}} = 0.01 M_{\odot}$.

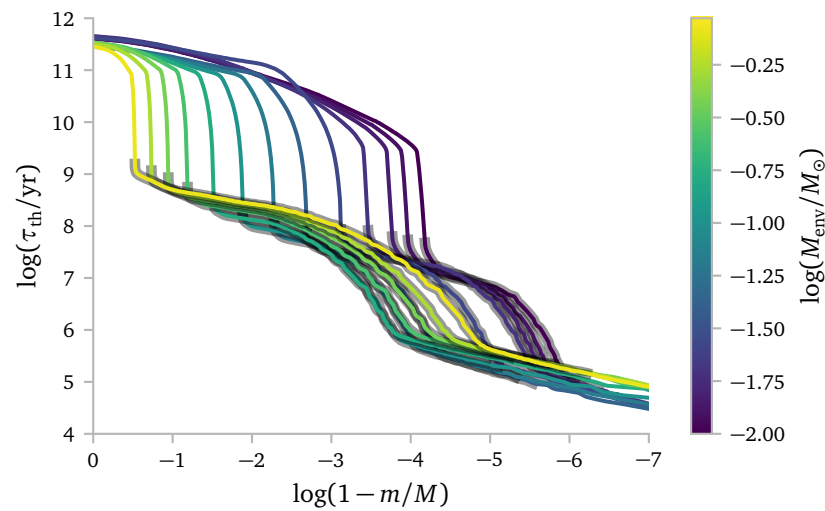
Here we can clearly see that the entropy profile of the star becomes a lot less flat once it has shed a lot of its envelope.

As a second test, I plot the radius of the profiles as a function of the envelope mass.

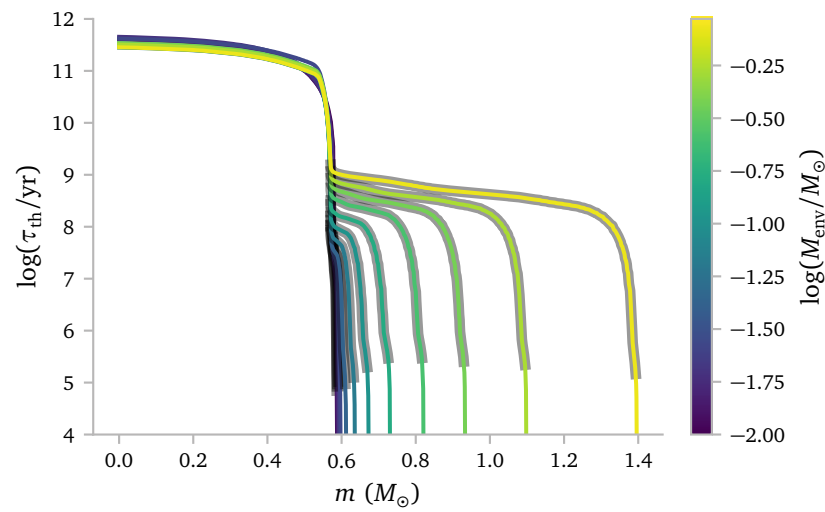


This plot not only shows the equal spacing of the profiles in log space, but also how there is not a simple relation between the envelope mass and the radius of the star. This is in part obviously caused by the thermal pulses.

Then, I plot the thermal time to the surface as a function of internal mass.



This plot aligns well with the results from Temmink et al. (2023). There, we see that an increased radius tends to shift the superadiabatic layers further from the surface. We see this here as well, with the $\log(M_{\text{env}}) \approx -1$ models having the deepest superadiabatic layers.



Here, we can quite clearly see that most of the envelope of the TPAGB stars is completely superadiabatic.

References

Karakas, A. I., Lattanzio, J. C., & Pols, O. R. (2002) *Parameterising the Third Dredge-up in Asymptotic Giant Branch Stars*. [PASA19, 515](#).

Karakas, A. I. & Lugaro, M. (2016) *Stellar Yields from Metal-rich Asymptotic Giant Branch Models*. [ApJ825, 26](#).

Paxton, B., Bildsten, L., Dotter, A., et al. (2011) *Modules for Experiments in Stellar Astrophysics (MESA)*. [ApJS192, 3](#).

Rees, N. R., Izzard, R. G., & Karakas, A. I. (2024) *Thermal pulses with MESA: resolving the third dredge-up*. [MNRAS527, 9643](#).

Sarmah, L., Bharat Kumar, Y., Campbell, S. W., Maben, S., & Reddy, B. E. (2026) *Unveiling the nature of barium stars. I. Asteroseismic masses and the evolutionary link between Ba dwarfs and giants*. [arXiv e-prints, arXiv:2606.25517](#).

Temmink, K. D., Pols, O. R., Justham, S., Istrate, A. G., & Toonen, S. (2023) *Coping with loss. Stability of mass transfer from post-main-sequence donor stars*. [A&A669, A45](#).

Vassiliadis, E. & Wood, P. R. (1993) *Evolution of Low- and Intermediate-Mass Stars to the End of the Asymptotic Giant Branch with Mass Loss*. [ApJ413, 641](#).